TP2: Accéléromètre et Gyroscope

ITI3: Capteurs

Robin Condat, Antoine Vaugeois

1 Objectifs de la séance

Aujourd'hui, nouveau capteur : accéléromètre, gyroscope et magnétomètre, tout en 1 ! Il paraît même qu'il peut faire le café... Au programme, on va rentrer dans le détail de la lectures des registres I2C du capteur, tracer de belles courbes et créer la nouvelle Wiimote !!

1.1 Matériel nécessaire

- Une Raspberry Pi 4
- Un MinIMU-9 v6 Gyro (`https://www.pololu.com/product/2862`)

1.2 Packages Python utilisés

- *smbus*: Pour communiquer avec des périphériques I2C
- *numpy*: Pour manipuler les données sous forme de tableaux
- *matplotlib*: Pour tracer des belles courbes :D
- *pandas / csv*: Pour enregistrer les données dans de beaux tableaux CSV
- *evdev*: Pour contrôler les touches directionnelles via un code python

2 Mise en place

Le capteur MinIMU-9 v6 a la particularité d'inclure 2 composants:

- Un accéléromètre et gyroscope LSM6DSO
- Un magnétomètre LIS3MDL

Dans ce capteur, les 2 composants communiquent séparément en I2C, à des adresses différentes ! De ce fait, il va falloir identifier à quelle adresse communique chaque composant du capteur.

- Connectez en I2C le capteur IMU MinIMU-9 v6 Gyro à la Raspberry Pi. **Si vous n'êtes pas 100 % sûr de votre branchement, faites-le vérifier par votre enseignant avant de mettre sous tension la Raspberry Pi !**
- Activez l'environnement virtuel python `env_capteurs`.
- Téléchargez le code disponible sur Moodle, décompressez l'archive zip et renseignez le chemin de votre dossier dans la variable d'environnement `PYTHONPATH` :

```
export PYTHONPATH=/chemin/vers/votre/dossier/TP:$PYTHONPATH
```

- Détectez les adresses des périphériques connectés en I2C (via *i2cdetect*)
- Identifiez pour chacun des composants l'adresse du registre `WHO_AM_I`, ainsi que la valeur stockée à ce registre (regardez dans les documentations)
- Interrogez pour chacune des adresses I2C le registre `WHO_AM_I` grâce à *i2cget*:

```
i2cget -y 1 <ADDR_I2C> <ADDR_REG>
```

- Modifiez le code `config/constants.py` en indiquant l'adresse I2C du LSM6DSO et du LIS3MDL

3 Comment le capteur communique?

A partir de maintenant, nous allons n'utiliser que l'accéléromètre et le gyroscope (donc le composant LSM6DSO), jusqu'à la fin du TP. Le capteur communique en I2C à une adresse précise (que vous avez identifiez précédemment). A cette adresse se trouve une zone mémoire de 256 octets, accessibles via 256 registres. Chaque registre (ou octet) peut être lu et ré-écrit. De ce fait, certains registres sont destinés à la configuration du capteur, d'autres à l'acquisition des données. Vous savez déjà qu'un registre sert à identifier le capteur (`WHO_AM_I`). A vous de comprendre ce que font les autres.

- Exécutez la commande suivante pour visualiser les registres disponibles à l'adresse I2C du capteur accéléromètre/gyroscope:

```
i2cdump -y 1 <ADDR_I2C>
```

- Sur combien de registres communique le composant ? Comparez avec la documentation (Register Mapping). Pour quelle(s) raison(s) voyez-vous une différence ?

4 Configuration du composant

Parmi ces registres, certains sont destinés à la configuration du composant. En écrivant des valeurs précises dans ces registres, on peut alors changer les paramètres d'acquisition. Ces registres ont souvent dans leur nom l'acronyme `CTRL` (pour contrôle). Vous pourrez constater que dans la documentation, il y a un grand nombre de registres de configuration, permettant de modifier de nombreux paramètres d'acquisition. Dans notre cas, nous allons nous concentrer uniquement sur 2 registres:

- `CTRL1_XL` pour configurer l'accéléromètre
- `CTRL2_G` pour configurer le gyroscope

Je vous invite donc à vous rendre aux pages décrivant ces deux registres, elles vous seront bien utiles !

- Dans le fichier `config/constants.py`, précisez les adresses des registres de configuration
- Quels paramètres de l'acquisition peuvent être modifiés via les registres `CTRL1_XL` et `CTRL2_G` ? Dans quelles unités ?
- Complétez les variables `CONFIG_XL_FREQ`, `CONFIG_XL_RANGE`, `CONFIG_G_FREQ` et `CONFIG_G_RANGE`, dans le fichier `config/constants.py`, afin de stocker les informations pour paramétrer les différentes configurations possibles
- Modifiez les fonctions `{accelerometer|gyroscope}_configuration` (dans les fichiers `sensors/IMU/{accelerometer|gyroscope}.py`) pour retourner la valeur hexadécimale à écrire dans le registre en fonction des arguments de la fonction
  Note : `LPF2_XL_EN`, `XL_HM_MODE`, `XL_FS_MODE` *et* `G_HM_MODE` *seront à zéros pour notre TP*

5 Lecture des données

Bon, c'est bien joli d'écrire dans des registres, mais on veut lire des données nous ! Ca tombe bien, c'est le moment. Après configuration, on peut lire les données d'acquisitions dans des registres spécifiques (ayant souvent l'acronyme `OUT` dans leur nom, comme dans output).

- Combien d'octets sont utilisés pour communiquer les mesures ? Quels sont les registres utilisés ?
- Dans le fichier `config/constants.py`, précisez les adresses des registres de sortie pour la communication des données.
  Note: Etant donné que les différents registres utilisés sont adjacents, on lit directement les données en tant que bloc, et non chaque octet séparément. Il suffit donc de préciser uniquement l'adresse du premier registre et le nombre d'octets à lire

La fonction de lecture de données (`read_raw_data.py` dans le fichier `sensors/IMU/imu.py`) nous renvoie ici des entiers signés (de -32768 à 32767). Ces valeurs numériques doivent être converties pour obtenir des mesures physiques.

- Trouvez dans la documentation (Mechanical Characteristics) les facteurs de conversion pour obtenir des valeurs physiques. Quels sont les paramètres d'acquisition qui influencent le facteur de conversion (parmi ceux qu'on a modifié) ? Quels sont les unités physiques des facteurs de conversion ?
- Complétez les variables `CONFIG_XL_SENSITIVITY` et `CONFIG_G_SENSITIVITY`, dans le fichier `config/constants.py`, afin de stocker les facteurs de conversion selon les ranges possibles du capteur
- Modifiez les fonctions `read_{accelerometer|gyroscope}` dans les fichiers `sensors/IMU/{accelerometer|gyroscope}.py` pour appliquer le bon facteur de conversion en fonction des paramètres de configuration

6 Enfin une première acquisition !

- Exécutez le programme *get_imu_data.py*, et laissez le capteur statique. Quels valeurs voyez-vous ? Cela est-il cohérent ?
- Bougez la plaque d'essai (modéremment !). Que constatez-vous au niveau des valeurs mesurées ?
- Modifiez les paramètres d'acquisition dans le fichier `config/parameters.py`. Comment cela impacte les mesures ?
- Enregistrez plusieurs acquisitions d'un même mouvement en modifiant les paramètres d'acquisition dans un fichier csv (option -s + nom de fichier csv lors de l'exécution du programme)
- Tracez les courbes des enregistrements via le code `postprocessing/plot_imu_data.py`. Comment les paramètres d'acquisition influencent les acquisitions ?

  ⇒ **Comment choisir les paramètres d'acquisitions selon le contexte ? Donnez plusieurs exemples et les paramètres associés que vous jugez être les plus adaptés, en justifiant ces choix**

7 GAMING TIME !

Allez, on va s'amuser ! Grâce à un accéléromètre et un gyroscope, on peut créer notre propre manette à l'instar de la Wiimote de Nintendo. Pour cela, on va mapper les touches directionnelles en fonction de l'inclinaison de la plaque d'essai.

- Installez le package python `evdev` dans votre environnement virtuel

- Exécutez la commande bash suivante :

```
sudo chmod +0666 /dev/uinput
```

- Modifiez le fichier `core/imu_controller.py` pour afficher la/les direction(s) en fonction des valeurs obtenues avec l'accéléromètre et/ou le gyroscope. Testez votre code avec un paramétrage des capteurs adéquat
- Modifiez votre code pour envoyer les instructions de touches directionnelles à la Raspberry Pi

```
send_key(key, press_or_release)
```

- Testez votre nouvelle manette avec un jeu en ligne utilisant 4 touches directionnelles